

Тема 5. Интерфейси (*продължение*). Шаблонни методи, класове и интерфейси. Колекции.

Имплементиране на интерфейси и наследяване

Пример 1: Имплементиране на интерфейси и наследяване

```
interface Alpha{
void set1(int n);
}

interface Beta{
void set2(int n);
}

class Gamma{
int number1;
void show(){
    System.out.println("Поле number1: "+number1);
}
}

class MyClass extends Gamma implements Alpha,Beta{
int number2;

public void set1(int n){
    number1=n;
}

public void set2(int n){
    number2=n;
}

void show(){
    super.show();
    System.out.println("Поле number2: "+number2);
}
}

class test1 {
public static void main(String[] args){

    Alpha A;
    Beta B;
```

Платформено независими програмни езици
Семинарно упражнение 5

```
MyClass obj=new MyClass();  
A=obj;  
B=obj;  
A.set1(100);  
B.set2(200);  
obj.show();  
}  
}
```

Изход в конзолата:

Поле number1: 100

Поле number2: 200

Чрез променливата *A* ще имаме достъп само до метод *set1*, а чрез променливата *B* – до метод *set2*.

Разширение на интерфейси (наследяване)

```
interface име extends интерфейс {  
    // деклариране на методи  
}
```

Пример 2: Разширение на интерфейси

```
interface Alpha2 {  
    int method1(int n);  
}  
  
interface Beta2 extends Alpha2 {  
    int method2(int n);  
}  
  
class MyClass2 implements Beta2 {  
  
    public int method1(int n) {  
        if(n==0) return 0;  
        else return n+method1(n-1);  
    }  
  
    public int method2(int n) {  
        if(n==1) {  
            return 0;  
        }  
        else if(n==2) {  
            return 1;  
        }  
    }  
}
```

```
    }  
    else {  
        return method2(n-1)+method2(n-2);  
    }  
}  
}  
class test2 {  
    public static void main(String[] args) {  
        MyClass2 obj=new MyClass2();  
        System.out.println("Метод1: "+obj.method1(9));  
        System.out.println("Метод2: "+obj.method2(9));  
    }  
}
```

Изход в конзолата:

Метод1: 45

Метод2: 21

Анонимен клас на основата на интерфейс

Създаване на обект на база анонимен клас, имплементиращ интерфейс

```
интерфейс променлива = new интерфейс() {  
    // дефиниране на методите на интерфейса  
}
```

Пример 3: Анонимен клас на основата на интерфейс

```
interface Alpha3 {  
    void set(int n);  
    int get();  
}  
  
class test3 {  
    public static void main(String[] args) {  
        Alpha3 A=new Alpha3() {  
            int number;  
  
            public void set(int n) {  
                number=n;  
            }  
            public int get() {  
                return number;  
            }  
        }  
    }  
}
```

```
};  
  
A.set(100);  
System.out.println("Поле: "+A.get());  
}  
}
```

Изход в конзолата:

Поле: 100

Важно! Обектът *A* се създава на база клас, който имплементира интерфейса *Alpha3* и поради тази причина може да бъде записана референция към обекта в интерфейлната променлива. Променлива *A* няма достъп до полето *number*, а само до методите, които са декларирани и имплементирани от интерфейса.

Шаблонни методи

Шаблонните методи се използват, когато искаме методът да бъде универсален и да не зависи от типа на данните. При описанието на шаблонните методи типът не се конкретизира, а се указва във вид на параметър, нар. шаблонен параметър. Действителният тип, който трябва да бъде използван на мястото на шаблонния параметър се определя въз основа на типа на аргумента. Само референтни типове (т.е. класове обвивки) мога да се използват за стойност на шаблонен параметър. Обикновено (но не е задължително) шаблонните методи са статични. Известни са още като генерици (generics).

Описание на шаблонен статичен метод

```
static <параметър> тип_на_резултата име_на_метода(аргументи){  
    // код на метода  
}
```

Когато се използват няколко шаблонни параметъра, те се разделят със запетая!

Пример 4: Шаблонни статични методи

```
class MyClass4{  
    int number;  
  
    MyClass4(int number){  
        this.number=number;  
    }  
    // Предефиниране на метод toString():  
    public String toString(){  
        return "MyClass4 "+number;  
    }  
}
```

```
}  
}  
  
class test4{  
static <T> void show(T[] arr){  
    for(int i=0;i<arr.length;i++){  
        System.out.print(arr[i]+ " ");  
    }  
    System.out.println();  
}  
static <T> String getText(T a,int n){  
    String res=n+": ";  
    res+=a;  
    return res;  
}  
  
public static void main(String[] args){  
  
    Integer[] arr={1,2,3,4,5,6};  
    Character[] symbols={'A','B','C','D'};  
    String[] text={"Alpha","Beta","Gamma"};  
  
    show(arr);  
    show(symbols);  
    show(text);  
    System.out.println(getText('A',1));  
    System.out.println(getText("Beta",2));  
    System.out.println(getText(10,3));  
  
    MyClass4 obj=new MyClass4(200);  
    System.out.println(getText(obj,4));  
}  
}
```

Изход в конзолата:

1 2 3 4 5 6

A B C D

Alpha Beta Gamma

1: A

2: Beta

3: 10

4: MyClass4 200

Стойностите на шаблонните параметри могат да бъдат определени в явен вид. В този случай в инструкцията за извикване на шаблонен метод пред името му в <> се поставят стойностите за шаблонните параметри.

Пример 5: Шаблонни статични методи с явно определяне на типа

```
class MyClass5 {  
    static <T> void show(T t) {  
        System.out.println("Стойност: "+t);  
    }  
}  
  
class test5 {  
    public static void main(String[] args) {  
        MyClass5.show(1);  
        MyClass5.<Integer>show(2);  
        MyClass5.show(1.0);  
        MyClass5.<Double>show(2.0);  
        MyClass5.show('A');  
        MyClass5.<Character>show('B');  
    }  
}
```

Изход в конзолата:

Стойност: 1

Стойност: 2

Стойност: 1.0

Стойност: 2.0

Стойност: A

Стойност: B

Шаблонните методи могат да бъдат дефинирани и в обикновени класове и да бъдат нестатични. Нестатичният метод се извиква от обект!

Пример 6: Шаблонни нестатични методи

```
class MyClass6 {  
    String name;  
    <X> void show(X arg) {  
        System.out.println(name+": "+arg);  
    }  
  
    MyClass6(String name) {  
        this.name=name;  
    }  
}
```

```
class test6{
public static void main(String[] args){
    MyClass6 A=new MyClass6("Обект A");
    MyClass6 B=new MyClass6("Обект B");

    A.show(1);
    A.show("Alpha");
    A.show('A');
    B.show(2);
    B.show("Beta");
    B.show('B');
}
}
```

Изход в конзолата:

Обект A: 1
Обект A: Alpha
Обект A: A
Обект B: 2
Обект B: Beta
Обект B: B

Шаблонен конструктор

Пример 7: Шаблонен конструктор

```
class Alpha7{
    int number;
    Alpha7(int number){
        this.number=number;
    }
    public String toString(){
        return "<Alpha7 "+number+">";
    }
}

class Beta7{
    String name;

    <T> Beta7(T name){
        this.name=name.toString();
    }
}

// Главен клас:
```

```
class test7{  
    public static void main(String[] args){  
        Beta7 A=new Beta7(333);  
        System.out.println(A.name);  
        Beta7 B=new Beta7("Java");  
        System.out.println(B.name);  
        Beta7 C=new Beta7(new Alpha7(300));  
        System.out.println(C.name);  
    }  
}
```

Изход в конзолата:

333

Java

<Alpha7 300>

Презареждане на шаблонни методи

Пример 8: Презареждане на шаблонни методи

```
class test8{  
    static <T> void show(T value){  
        System.out.println("[1] "+value);  
    }  
    // Метод с текстов аргумент:  
    static void show(String str){  
        System.out.println("[2] Текст "+str);  
    }  
    // Шаблонен метод с два параметъра:  
    static <T,U> void show(T t,U u){  
        System.out.println("[3] "+t+" и "+u);  
    }  
    // Шаблонен метод с един параметър и два аргумента:  
    static <T> void show(T t,int n){  
        System.out.println("[4] "+t+" и число "+n);  
    }  
    // Метод с два аргумента (число и символ):  
    static void show(int n,char s){  
        System.out.println("[5] Число "+n+" и символ "+s);  
    }  
    // Главен метод:  
    public static void main(String[] args){  
        show(100);  
        show('A');  
        show("Alpha");  
    }  
}
```

```
show("Bravo","Charlie");  
show('B',200.0);  
show("Delta",300);  
show(400,'C');           //!  
test8.<Character,Integer>show('D',500);  
test8.<Character>show('D',500);  
}  
}
```

Изход в конзолата:

```
[1] 100  
[1] A  
[2] Текст Alpha  
[3] Beta и Gamma  
[3] B и 200.0  
[4] Delta и число 300  
[5] Число 400 и символ C  
[3] D и 500  
[4] D и число 500
```

Шаблонни класове

В шаблонния клас типът данни се явява параметър. В тялото на класа тези параметри се използват като идентификатори на типа.

Дефиниране на шаблонен клас

```
class име_на_класа<параметри>{  
    // описание на класа  
}
```

Създаване на обект на базата на шаблонен клас

```
клас<стойности> променлива = new  
клас<стойности> (аргументи);  
  
или  
клас<стойности> променлива = new клас<> (аргументи);
```

Пример 9: Шаблонни класове

//Шаблонен клас с един параметър:

```
class Alpha9<T>{
```

// Поле от шаблонен тип:

```
T value;
```

// Конструктор:

Платформено независими програмни езици
Семинарно упражнение 5

```
Alpha9(T value){
    this.value=value;
}
// Метод:
void show(){
    System.out.println("<Alpha9 "+value+">");
}
}
//Шаблонен клас с два параметъра:
class Beta9<T,U>{
    // Полета от шаблонен тип:
    T first;
    U second;
    // Конструктор:
    Beta9(T first,U second){
        this.first=first;
        this.second=second;
    }
    // Метод:
    void show(){
        System.out.println("<Beta9 "+first+" "+second+">");
    }
}
//Главен клас:
class test9{
    public static void main(String[] args){
        // Създаване на обекти на основата на шаблонни класове:
        Alpha9<Integer> A1=new Alpha9<Integer>(123);
        Alpha9<String> A2=new Alpha9<>("Red");
        Beta9<Integer,String> B1;
        B1=new Beta9<Integer,String>(321,"Green");
        Beta9<Character,Character> B2=new Beta9<>('B','C');
        // Проверка на резултата:
        A1.show();
        A2.show();
        B1.show();
        B2.show();
    }
}
```

Изход в конзолата:

```
<Alpha9 123>
<Alpha9 Red>
<Beta9 321 Green>
<Beta9 B C>
```

Шаблонни интерфейси

На основата на шаблонен интерфейс могат да бъдат създавани както обикновени, така и шаблонни класове.

Пример 10: Шаблонен интерфейс

```
//Шаблонен интерфейс:
interface MyInterface10<T>{
void show(T t);
}
//Шаблонен клас:
class Alpha10<T,U> implements MyInterface10<T>{
    U value;
public void show(T t){
        System.out.println(t+": "+value);
    }
}
//Обикновен клас:
class Beta10 implements MyInterface10<String>{
public void show(String t){
    System.out.println("Текст: "+t);
}
}
//Главен клас:
class test10{
public static void main(String[] args){
    Alpha10<Character,String> A=new Alpha10<>();
    A.value="Alpha";
    A.show('A');
    Beta10 B=new Beta10();
    B.show("Beta");
}
}
```

Изход в конзолата:

A: Alpha

Текст: Beta

Шаблонни класове и наследяване

На основата на шаблонен клас може да бъде създаден както обикновен клас, така и шаблонен клас.

Пример 11: Шаблонен клас и наследяване

```
//Шаблонен суперклас:
class MyClass11<T,U>{
// Полета:
T value;
U code;
// Методи:
void set(T a,U b){
    value=a;
    code=b;
}
void show(){
    System.out.println("[1] "+value);
    System.out.println("[2] "+code);
}
}

//Шаблонен подклас:
class Alpha11<T> extends MyClass11<T,String> {}

//Обикновен подклас:
class Beta11 extends MyClass11<Character,Integer> {}

//Главен клас:
class test11 {
public static void main(String[] args){
    MyClass11<String,Character> obj=new MyClass11<>();
    obj.set("MyClass",'K');
    obj.show();
    Alpha11<Double> A=new Alpha11<>();
    A.set(111.0,"Alpha");
    A.show();
    Beta11 B=new Beta11();
    B.set('F',333);
    B.show();
}
}
```

Изход в конзолата:

```
[1] MyClass
[2] K
[1] 111.0
[2] Alpha
[1] F
[2] 333
```

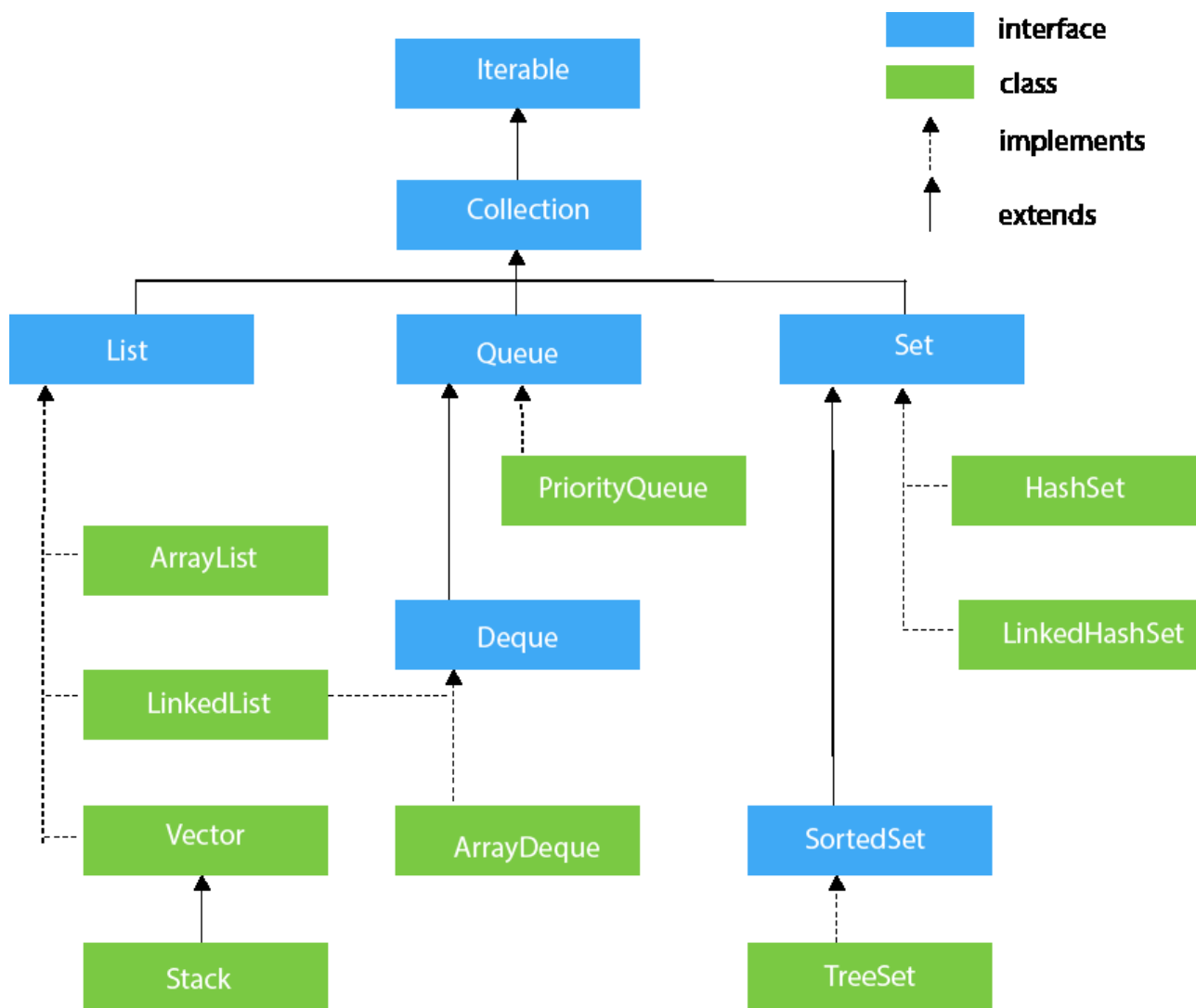
Колекции

Колекцията е организирана по специален начин група обекти. Интерфейсите определят стандартите за работа с колекции. Колекцията е обект, създадена на базата на клас, имплементиращ интерфейса **Collection**.

Таблица 1: Методи на интерфейса Collection

No.	Method	Description
1	public boolean add(E e)	It is used to insert an element in this collection.
2	public boolean addAll(Collection<? extends E> c)	It is used to insert the specified collection elements in the invoking collection.
3	public boolean remove(Object element)	It is used to delete an element from the collection.
4	public boolean removeAll(Collection<?> c)	It is used to delete all the elements of the specified collection from the invoking collection.
5	default boolean removeIf(Predicate<? super E> filter)	It is used to delete all the elements of the collection that satisfy the specified predicate.
6	public boolean retainAll(Collection<?> c)	It is used to delete all the elements of invoking collection except the specified collection.
7	public int size()	It returns the total number of elements in the collection.
8	public void clear()	It removes the total number of elements from the collection.
9	public boolean contains(Object element)	It is used to search an element.
10	public boolean containsAll(Collection<?> c)	It is used to search the specified collection in the collection.
11	public Iterator iterator()	It returns an iterator.
12	public Object[] toArray()	It converts collection into array.
13	public <T> T[] toArray(T[] a)	It converts collection into array. Here, the runtime type of the returned array is that of the specified array.
14	public boolean isEmpty()	It checks if collection is empty.
15	default Stream<E> parallelStream()	It returns a possibly parallel Stream with the collection as its source.
16	default Stream<E> stream()	It returns a sequential Stream with the collection as its source.
17	default Spliterator<E> spliterator()	It generates a Spliterator over the specified elements in the collection.
18	public boolean equals(Object element)	It matches two collections.
19	public int hashCode()	It returns the hash code number of the collection.

Шаблонният интерфейс List разширява интерфейс Collection и е предназначен за реализация на колекция от тип списък (подредена последователност от обекти, към които може да бъде получен достъп по индекс). Съществуват и други интерфейси за работа с колекции, които разширяват интерфейса Collection и се имплементират в класовете, на базата на които се създават колекции от обекти. Йерархията им е показана на фиг. 1.



Фиг. 1. Йерархия на Collection framework¹

Класът ArrayList се използва за реализация на динамични списъци, които да променят размерите си в процеса на изпълнение на програмата.

Пример 12: Динамичен списък

```
//Импортиране на класа:
import java.util.ArrayList;
//Главен клас:
class test12{
public static void main(String[] args){
    // Създаване на празен списък:
    ArrayList<Integer> nums=new ArrayList<>();
    // Показване съдържанието на списъка:
    System.out.println(nums);
    // Размер на списъка:
```

¹ <https://www.javatpoint.com/collections-in-java>

Платформено независими програмни езици
Семинарно упражнение 5

```
System.out.println("Размер: "+nums.size());
// Добавяне на елемент в списъка:
nums.add(100);
nums.add(200);
nums.add(300);
// Показване съдържанието на списъка:
System.out.println(nums);
// Размер на списъка:
System.out.println("Размер: "+nums.size());
// Добавяне на елементи в списъка:
nums.add(0,-1);
nums.add(2,0);
// Показване съдържанието на списъка:
System.out.println(nums);
// Размер на списъка:
System.out.println("Размер: "+nums.size());
// Замяна на елемент в списъка:
nums.set(1,123);
// Съдържание на списъка:
System.out.println(nums);
// Изтриване на елемент от списъка:
nums.remove(2);
// Съдържание на списъка:
System.out.println(nums);
// Размер на списъка:
System.out.println("Размер: "+nums.size());
// Изчистване на списъка:
nums.clear();
// Показване съдържанието на списъка:
System.out.println(nums);
// Размер на списъка:
System.out.println("Размер: "+nums.size());
// Ако списъкът е празен:
if(nums.isEmpty()) nums.add(321);
// Показване съдържанието на списъка:
System.out.println(nums);
// Размер на списъка:
System.out.println("Размер: "+nums.size());
}
}
```

Изход в конзолата:

[]

Размер: 0

[100, 200, 300]

Платформено независими програмни езици
Семинарно упражнение 5

Размер: 3

`[-1, 100, 0, 200, 300]`

Размер: 5

`[-1, 123, 0, 200, 300]`

`[-1, 123, 200, 300]`

Размер: 4

`[]`

Размер: 0

`[321]`

Размер: 1

Платформено независими програмни езици
Семинарно упражнение 5

Задача за самостоятелна работа